

Thrashing and Working Set Model

Memory Management – Virtual Memory

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

By the end of this topic, students should be able to:

- Explain the concept of thrashing and its causes.
- Relate thrashing to the degree of multiprogramming and locality.
- Define the working set of a process.
- Calculate the working-set size for a given reference string.
- Use the working-set model to control thrashing.

Thrashing

Thrashing occurs when a process spends more time handling page faults and swapping pages than executing useful instructions.

Typical situation

The total number of frames required by active processes exceeds the available physical frames.

$$\sum_{i=1}^n WSS_i > m$$

where

- WSS_i = working-set size of process i
- m = total number of available physical frames

Effect

High page-fault rate $\rightarrow$ high disk I/O $\rightarrow$ low CPU utilization $\rightarrow$ poor system performance.

Why Does Thrashing Occur?

- ① A process needs more pages than the frames currently allocated to it.
- ② A page fault occurs frequently.
- ③ The OS brings the required page from secondary storage.
- ④ Another required page may be evicted immediately.
- ⑤ The cycle repeats continuously.

Important observation

Increasing the degree of multiprogramming does **not** always increase CPU utilization. Beyond a certain point, it can cause thrashing.

Too many active processes $\Rightarrow$ insufficient frames per process $\Rightarrow$ thrashing

Working Set Model

The **working set** of a process is the set of distinct pages referenced during the most recent Δ page references.

Definition

$$W(t, \Delta) = \{\text{pages referenced during the last } \Delta \text{ references at time } t\}$$

- Δ is called the **working-set window**.
- $WSS(t, \Delta) = |W(t, \Delta)|$ is the working-set size.
- The working set represents the process's current **locality**.

Key idea

A process should have enough frames to contain its current working set.

Working Set Example 1 – Basic

Reference string:

1, 2, 3, 1, 4, 2, 5, 1, 2, 3

Working-set window:

$\Delta = 4$			
t	Reference	Last 4 References	WSS
1	1	1	1
2	2	1,2	2
3	3	1,2,3	3
4	1	1,2,3,1	3
5	4	2,3,1,4	4
6	2	3,1,4,2	4
7	5	1,4,2,5	4
8	1	4,2,5,1	4
9	2	2,5,1,2	3
10	3	5,1,2,3	4

$$\max(WSS) = 4,$$

$$\text{Average WSS} = 3.2$$

Example 1 – Interpretation

$$WSS_{\max} = 4$$

Therefore, at least 4 frames are required if the process is to keep its entire current working set in memory.

Observation

The working-set size changes over time because the process moves between different localities.

For example:

$$W(7, 4) = \{1, 4, 2, 5\}$$

so

$$WSS(7, 4) = 4.$$

Later:

$$W(9, 4) = \{2, 5, 1\}$$

so

$$WSS(9, 4) = 3$$

Example 2 – GATE-Type Calculation

Consider the page-reference string:

$c, c, d, b, c, e, c, e, a, d$

Working-set window:

$$\Delta = 4$$

Assume initially:

$$W(0, 4) = \{a, d, e\}.$$

Determine:

- 1 Total number of page faults.
- 2 Average number of frames required.

Example 2 – GATE-Type Calculation

Consider the page-reference string:

c, c, d, b, c, e, c, e, a, d

Working-set window:

$$\Delta = 4$$

Assume initially:

$$W(0, 4) = \{a, d, e\}.$$

Method

At each reference:

- 1 Consider the current reference and previous $\Delta - 1$ references.
- 2 Remove references outside the window.
- 3 Count only distinct pages.
- 4 Compare the current page with the resident working set.

Example 2 – Solution

t	Ref.	Working Set	WSS	Fault
1	c	{e,d,a,c}	4	Yes
2	c	{d,a,c}	3	No
3	d	{a,c,d}	3	No
4	b	{c,d,b}	3	Yes
5	c	{d,b,c}	3	No
6	e	{d,b,c,e}	4	Yes
7	c	{b,c,e}	3	No
8	e	{c,e}	2	No
9	a	{c,e,a}	3	Yes
10	d	{c,e,a,d}	4	Yes

Page faults = 5

$$\text{Average frames} = \frac{4 + 3 + 3 + 3 + 3 + 4 + 3 + 2 + 3 + 4}{10} = 3.2$$

Example 3 – Detecting Thrashing

Suppose a system has 10 physical frames and three active processes:

$$WSS_1 = 4, \quad WSS_2 = 5, \quad WSS_3 = 3$$

Total working-set demand:

$$D = \sum WSS_i = 4 + 5 + 3 = 12$$

Available frames:

$$m = 10$$

Therefore:

$$D > m$$

Conclusion

The system cannot accommodate all three working sets simultaneously. Thrashing is likely to occur.

OS action

Example 3 – Detecting Thrashing

Conclusion

The system cannot accommodate all three working sets simultaneously. Thrashing is likely to occur.

OS action

Reduce the degree of multiprogramming, for example by suspending one process and reclaiming its frames.

Working Set Model and Thrashing Control

For n processes:

$$D = \sum_{i=1}^n WSS_i$$

Compare D with the total available frames m .

Condition	OS interpretation
$D < m$	Enough frames; processes can coexist
$D \approx m$	High frame pressure; monitor carefully
$D > m$	Thrashing likely; reduce multiprogramming

Main principle

The OS tries to ensure:

$$\sum_{i=1}^n WSS_i \leq m$$

Thrashing: Symptoms and Remedies

Symptoms

- Very high page-fault rate
- Excessive disk I/O
- Low CPU utilization
- Poor response time and throughput

Remedies

- Reduce degree of multiprogramming.
- Suspend one or more processes.
- Allocate sufficient frames to active processes.
- Use the working-set model to estimate frame demand.
- Page-Fault Frequency (PFF) can also be used as a control mechanism.

Exam / GATE Quick Questions

① If $\Delta = 5$, how many page references are considered when calculating the working set?

② If

$$WSS_1 = 5, WSS_2 = 4, WSS_3 = 3, \quad m = 10,$$

is thrashing likely?

③ Why can increasing the degree of multiprogramming reduce CPU utilization?

④ For the reference string

1, 2, 1, 3, 2, 4

and $\Delta = 3$, calculate the working set at $t = 6$.

Answers

- Q1: 5 references.
- Q2: Yes, because $5 + 4 + 3 = 12 > 10$.
- Q3: Processes receive fewer frames, increasing page faults and causing thrashing.
- Q4: $W(6, 3) = \{2, 4\}$, so $WSS = 2$.

- **Thrashing**: excessive paging with very little useful CPU work.
- Main cause: insufficient frames for the active processes.
- **Working Set** captures the pages currently used by a process.
- Working-set size:

$$WSS(t, \Delta) = |W(t, \Delta)|$$

- Total frame demand:

$$D = \sum WSS_i$$

- To avoid thrashing:

$$D \leq m$$

- If $D > m$, reduce the degree of multiprogramming.